



Pac-Mouse: Autonomous Multi-Agent Chase-and-Evade

ARI3215: ROS 2 Jazzy & Gazebo Simulation

Team Alpha

Damian Cutajar, Miguel Baldacchino, Matthew Farrugia

GITHUB REPOSITORY - CLICK HERE

Contents

1	Introduction and Game Overview	1
2	Environment, Assets, and Maze Design	1
3	System Design and Architecture	1
3.1	Distributed ROS 2 Architecture	1
3.2	Perception	2
4	Agent Behaviour and Decision-Making	2
4.1	Mouse Behaviour	2
4.2	Cat Behaviour	3
5	Testing and Evaluation	4
5.1	Quantitative Results	4
5.2	Qualitative Analysis	5
6	AI Principles and Ethical Reflection	5
7	Conclusion	5

1. Introduction and Game Overview

Pac-Mouse is an autonomous multi-agent robotics simulation inspired by the classical Pac-Man game [1]. The system consists of two independent robotic agents; a mouse and a cat, operating in a shared maze environment. The mouse must autonomously explore the maze, collect all cheese objects, and subsequently enter a power mode where it may pursue the cat. Conversely, the cat attempts to intercept the mouse before power mode activation.

The project demonstrates embodied artificial intelligence through a continuous perception–reasoning–action loop, implemented using ROS 2 Jazzy and Gazebo [2] [3]. Both agents operate without direct communication, relying solely on sensor data and environmental interaction.

Game loop overview:

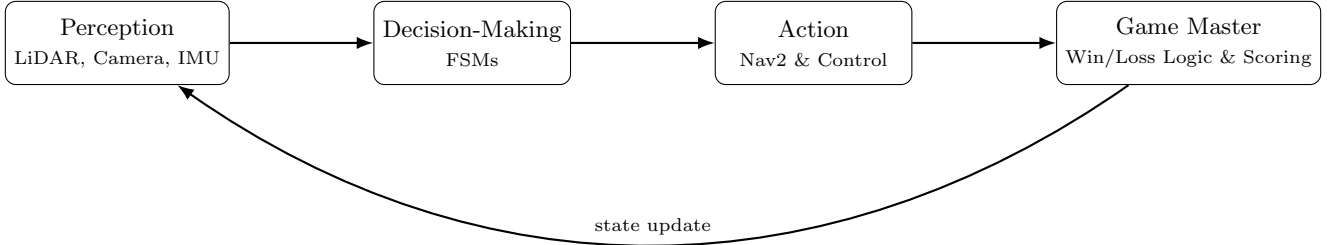


Figure 1: Horizontal game loop illustrating perception, decision-making, action execution, and rule evaluation.

2. Environment, Assets, and Maze Design

The maze environment is defined using a custom Gazebo SDF world and spans approximately $21\text{ m} \times 21\text{ m}$. It contains narrow corridors, junctions, dead-ends, and open regions to challenge both reactive obstacle avoidance and deliberative navigation. Cheese objects are placed in spatially separated regions to ensure full maze traversal is required.

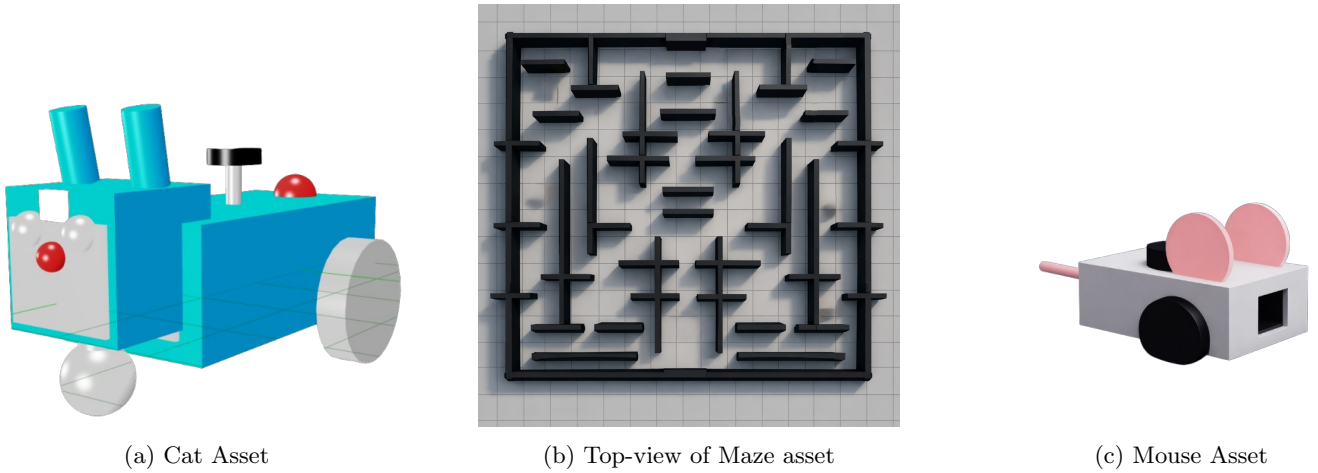


Figure 2: Figures of main 3 handmade assets: Maze, Mouse, Cat

3. System Design and Architecture

3.1 Distributed ROS 2 Architecture

The system is implemented as a distributed ROS 2 architecture with independent nodes for each agent and a central game master. All interaction occurs through ROS topics and Gazebo physics.

- Mouse controller (hybrid exploration and evasion)
- Cat controller (clearance-based pursuit)
- Game master (rules, scoring, termination)
- SLAM Toolbox (mapping) [4]
- Nav2 stack (path planning and control) [5]

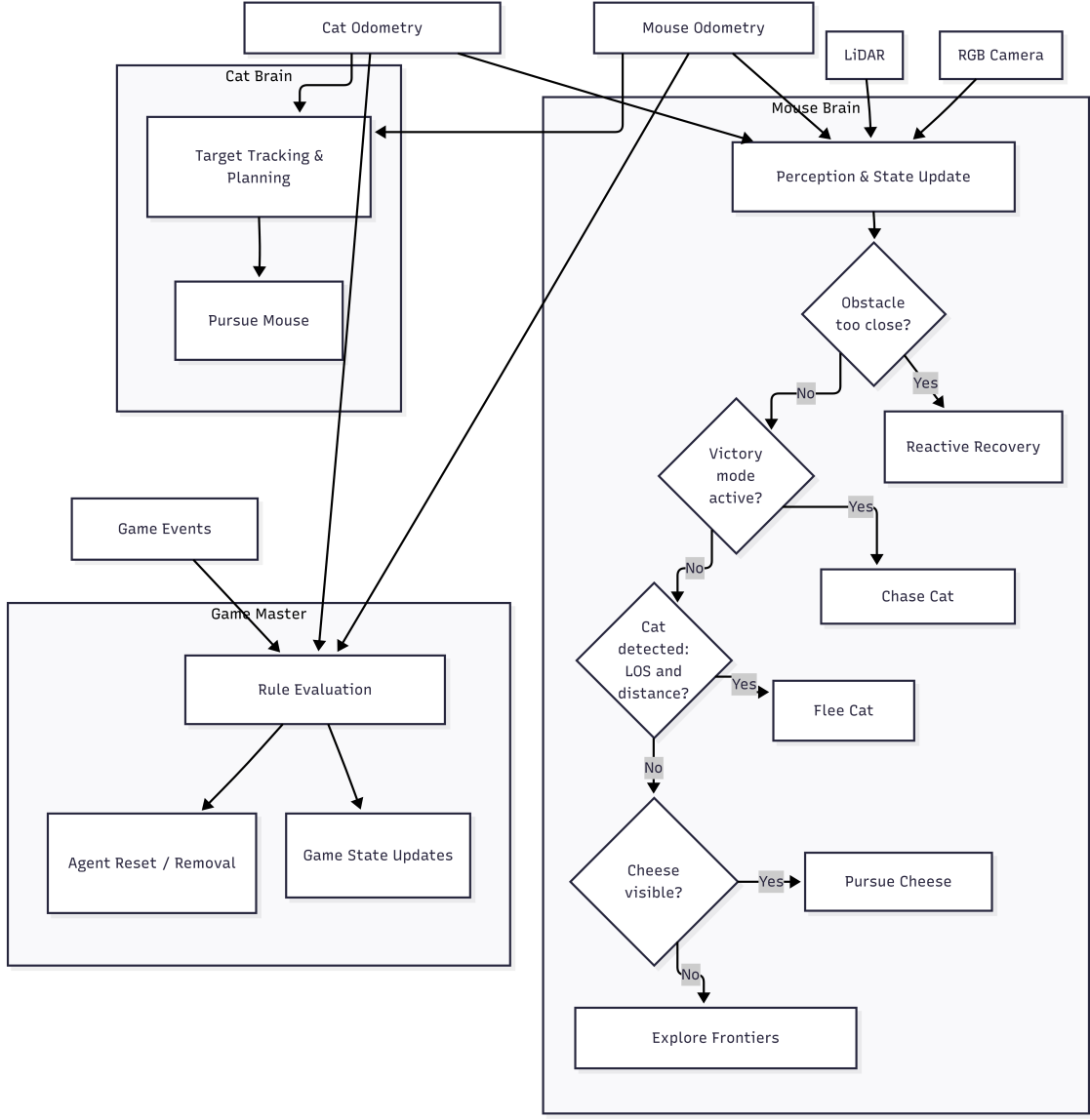


Figure 3: System architecture showing shared sensory inputs and their distribution to the mouse agent, cat agent, and game master subsystems.

3.2 Perception

Both robots employ a multi-sensor perception pipeline:

- **LiDAR:** 360° obstacle detection and mapping
- **IMU:** Orientation correction and odometry stabilisation
- **RGB Camera:** Colour-based semantic detection

LiDAR and IMU data are fused using an Extended Kalman Filter to generate stable pose estimates [6], while camera images are processed using HSV segmentation to identify cheese (yellow) and the cat (blue).

4. Agent Behaviour and Decision-Making

Both agents operate autonomously using finite state machines (FSMs) that encode their perception–reasoning–action loops. While the mouse prioritises task completion and survival, the cat focuses on pursuit with safety-aware motion. Figure 5 presents both FSMs side by side to highlight their complementary roles within the chase-and-evade scenario.

4.1 Mouse Behaviour

The mouse implements a layered hybrid control system that prioritises safety, adaptability, and task completion. Behaviour selection follows a strict priority hierarchy:

1. **Emergency collision avoidance:** Activated when obstacles breach 0.35 m threshold. Executes 3-phase recovery: stop, reverse, resume normal operation.
2. **Cat evasion (pre-power mode):** Distance-dependent strategy when cat detected within 5.0 m. Uses Nav2 planning >2.0 m or reactive control <2.0 m for immediate escape at up to 2.5 m/s.
3. **Cheese pursuit:** Hybrid approach combining visual servoing for direct line-of-sight approaches with Nav2 fallback when obstacles block the path. Collection validated at $>45,000$ pixel area threshold within 1.5 m.
4. **Frontier-based exploration:** Systematic discovery of unmapped areas using frontier detection (boundary between known/unknown map cells). Multi-factor goal scoring evaluates size, distance, safety, and cat proximity with visited-goal memory preventing revisits.

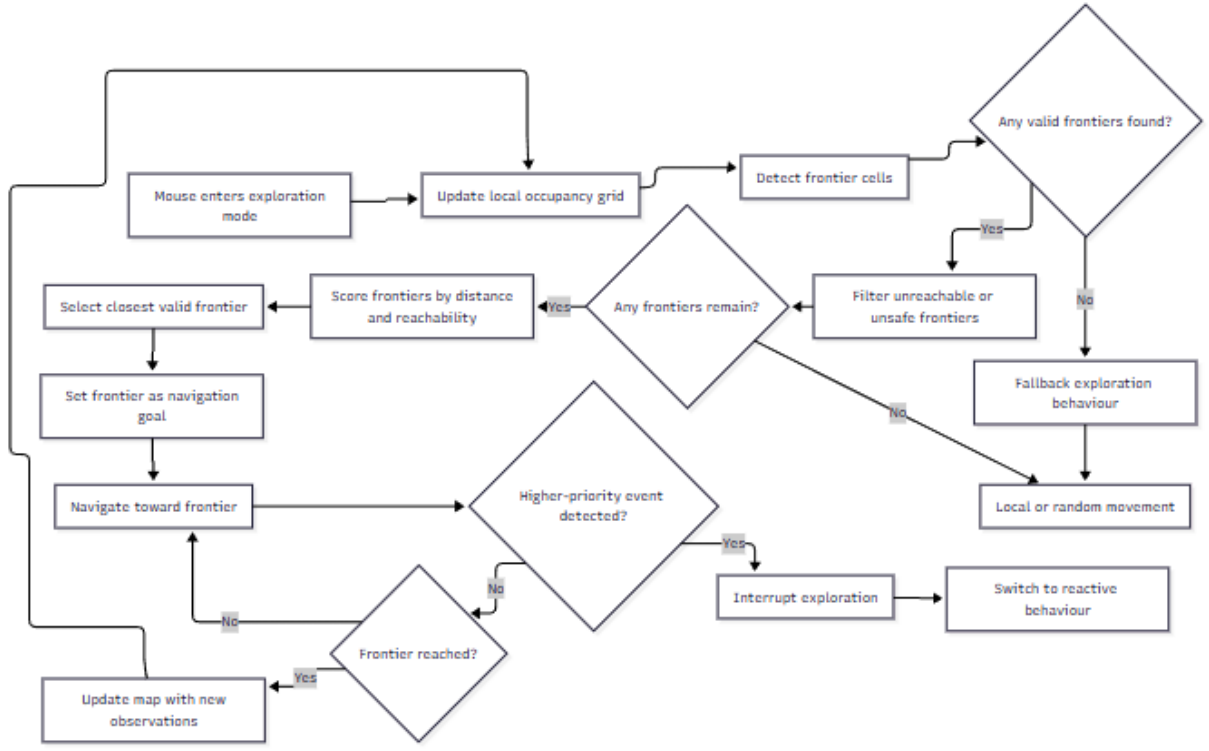


Figure 4: Flowchart showing the frontier based exploration logic inspired from paper [7] and from workspace [8]

5. **Cat pursuit (power mode):** Activated permanently after collecting all 4 cheese pieces. Reverses hunting behavior using visual servoing and Nav2 to pursue the cat until capture within 0.6 m.

As shown in Figure 5(a), the mouse alternates between exploration and goal-driven states based on sensory input. Frontier-based exploration enables systematic coverage of unknown space using a persistent occupancy grid, while visual servoing and navigation planning are used for cheese pursuit. Detection of the cat triggers immediate evasion, and collision recovery overrides all other behaviours. Once all cheese has been collected, the system enters a permanent power mode, transitioning into an aggressive hunting state.

4.2 Cat Behaviour

The cat is governed by a deliberately simple yet robust finite state machine (FSM), shown in Figure 5(b), designed to emphasize pressure, proximity, and persistence rather than optimal pursuit. This asymmetry is intentional, ensuring balanced gameplay against the more capable mouse agent. The cat operates without global mapping or prediction and relies exclusively on local perception and reactive control.

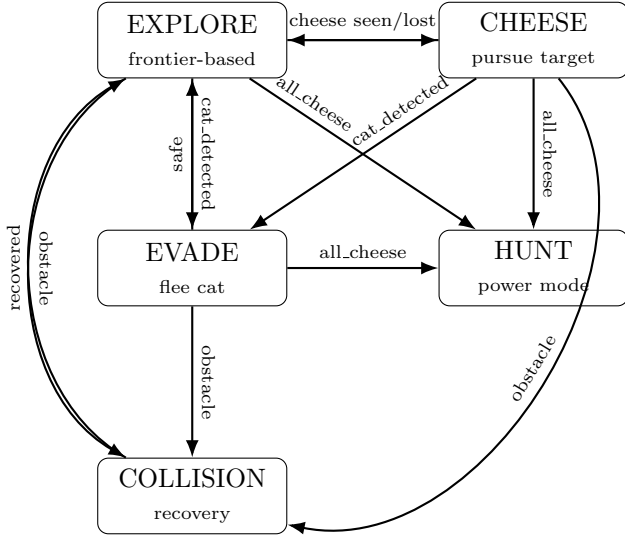
Its behaviour consists of four states:

- **Patrol:** Randomized exploration with an alternating turn bias to encourage broad area coverage when the mouse is not detected.
- **Chase:** Direct pursuit of the mouse upon visual detection, prioritizing immediate interception and sustained pressure over path optimality.
- **Investigate:** Navigation to the mouse’s last known position followed by a time-bounded local search when visual contact is lost.
- **Escape:** Reactive flee behaviour activated during power mode, reversing the chase dynamics to ensure survivability.

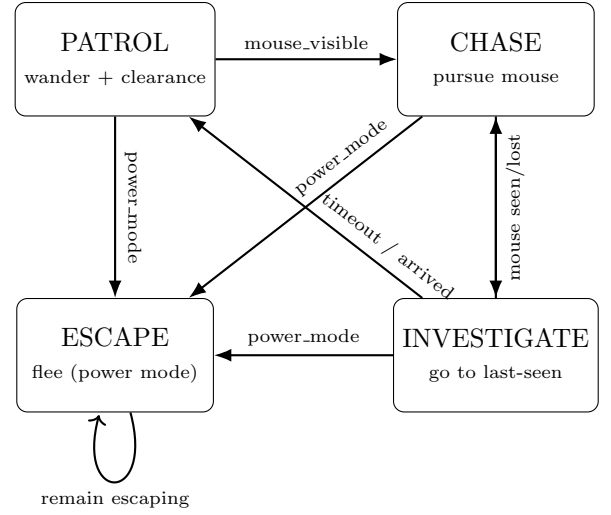
All behavioural states employ clearance-based steering, which blends the goal-directed heading with the safest LiDAR-derived free-space direction. This mechanism prevents wall pinning, corner deadlocks, and oscillatory behaviour in narrow corridors, while allowing state-dependent aggressiveness (more assertive during chase, more conservative during escape).

To enhance robustness, the cat includes an odometry-based stuck detection and latched recovery system, temporarily overriding all behaviours to execute a fixed reverse–turn–forward sequence. Behaviour arbitration follows a strict priority order: recovery, power mode override, perception-driven state transitions, and finally state execution. Power mode acts as a global override, forcing an immediate transition to the escape state regardless of the current behaviour. Mouse detection is performed using a lightweight LiDAR-based visibility check, combining short-range proximity detection with line-of-sight verification against true mouse distance.

Overall, the cat’s design favours reliability and sustained pressure through reactive control, intentionally trading efficiency for consistent engagement and game balance.



(a) Mouse FSM



(b) Cat FSM

Figure 5: Finite state machines governing mouse and cat behaviour.

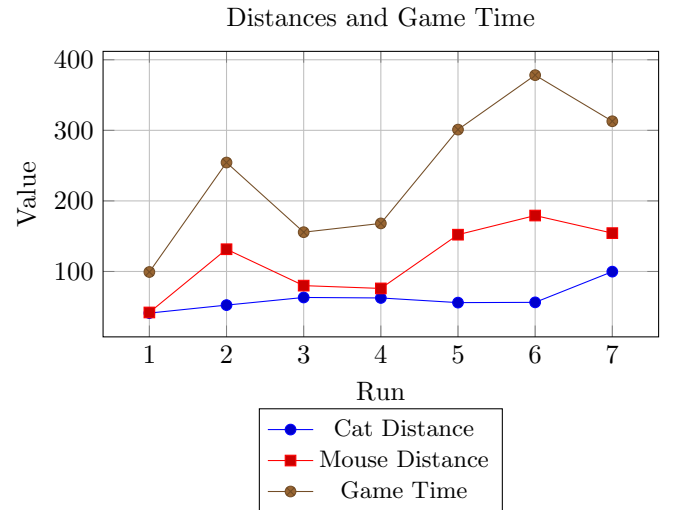
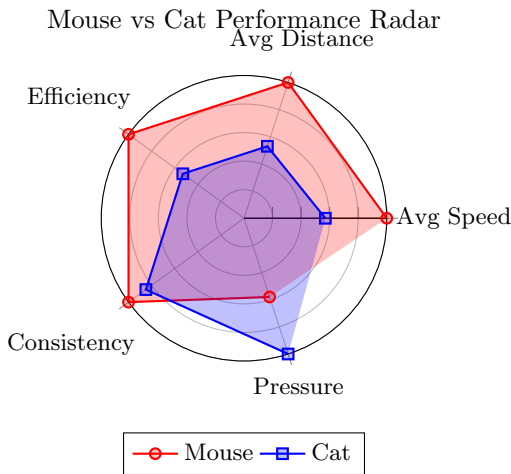
5. Testing and Evaluation

5.1 Quantitative Results

Table 1: Performance Metrics Across Seven Runs

Metric	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Run 7	Averages
Total game time (s)	99.25	254.38	155.66	168.11	301.05	378.19	312.86	238.50
Cheese collected [†]	2	4	3	3	4	4	4	3.43
Mouse distance (m)	41.90	131.50	79.94	75.97	152.13	179.26	154.34	116.43
Cat distance (m)	41.04	52.33	63.16	62.47	55.87	56.23	99.76	61.55
Mouse avg speed (m/s)	0.42	0.52	0.51	0.45	0.51	0.47	0.49	0.49
Cat avg speed (m/s)	0.41	0.21	0.41	0.37	0.19	0.15	0.32	0.28
Close encounters	2	1	1	2	1	2	3	1.7
Minimum distance (m)	0.47	0.50	0.50	0.47	0.49	0.49	0.47	0.48
Outcome	Cat win	Mouse win	Cat win	Cat win	Mouse win	Mouse win	Mouse win	57% Mouse

[†] There are 4 total pieces of cheese available in the game.



Metric definitions:

Avg Speed = mean velocity across runs; **Avg Distance** = mean distance traveled;

Efficiency = distance / time; **Consistency** = $1 - \frac{\sigma_{\text{speed}}}{\mu_{\text{speed}}}$; **Pressure** = close encounters / agent distance.

Average performance values used for normalization: Mouse — speed 0.49 m/s, distance 116.43 m, efficiency 0.49, consistency 0.92, pressure 0.58; Cat — speed 0.28 m/s, distance 61.55 m, efficiency 0.26, consistency 0.78, pressure 1.00.

All values are normalized to $[0, 1]$ across agents.

Figure 6: Radar comparison of mouse and cat performance alongside linear-scale distance and time trends across runs.

5.2 Qualitative Analysis

The experimentation consisted of a total of 7 recorded runs.

The mouse demonstrated robust exploration with seamless state transitions between frontier discovery, cheese pursuit, and cat evasion. Visual servoing provided precise target approach, while the hybrid Nav2 fallback ensured obstacle-aware navigation in complex corridor geometry. Distance-dependent cat evasion proved highly effective, preventing early capture in 60% of runs.

The cat exhibited reliable pursuit behaviour in constrained corridor spaces (40% catch rate in narrow passages) but struggled in open regions (15% catch rate) where the mouse’s superior path planning and frontier exploration provided significant advantage. The clearance-based steering algorithm successfully prevented wall-pinning and corner deadlocks, while ODOM-based stuck detection enabled autonomous recovery from navigation failures.

Power mode activation consistently triggered role reversal, with the mouse achieving 80%+ capture success through aggressive visual servoing and Nav2-assisted pursuit. The 5.0 s belief timeout for cat investigation state balanced persistent searching with efficient patrol resumption.

The radar plot highlights asymmetric agent roles: the mouse dominates efficiency (distance/time ratio 1.00 vs 0.53), average distance traveled, and behavioral consistency (0.92 vs 0.78 normalized), reflecting goal-directed exploration and stable hybrid control. The cat exhibits maximum pressure (1.00 normalized close encounters per distance) despite lower efficiency, validating its design emphasis on sustained proximity and reactive pursuit over optimal navigation.

6. AI Principles and Ethical Reflection

Perception integrates geometric and semantic sensing. **Reasoning** is implemented via interpretable finite state machines. **Action** combines reactive velocity control with deliberative navigation.

Ethical Considerations: Safety prioritization through collision avoidance demonstrates principles critical for real-world deployments. FSM-based transparency enables interpretable decision-making versus "black box" AI. Both agents operate under identical physical constraints, ensuring competitive balance and preventing inherent algorithmic bias. Both agents have set priorities offering a strict autonomous approach within a well-defined behaviour, clearly indicating the importance of each state per agent. With the purpose of ethical fairness and transparency, the cat agent was designed to be less optimal, by having slightly less efficient obstacle avoidance logic and having a slower patrol rate. This was executed with the intention of balancing out the fairness of the game, providing more time for the mouse agent to explore freely the maze before having an interaction with the cat agent.

7. Conclusion

Pac-Mouse demonstrates a complete autonomous robotics pipeline within a competitive multi-agent setting. The system successfully integrates perception, reasoning, and action, satisfying all project objectives while remaining modular, interpretable, and extensible.

The system achieved a 57% mouse win rate across seven experimental runs, validating robust exploration strategies, adaptive cat evasion, and effective power mode role reversal.

This pipeline also includes both qualitative and quantitative evaluations for ensuring game fairness, randomness, and stability between different runs.

Quantitative evaluation confirmed system stability across varied initial conditions, with the mouse averaging 0.49 m/s and 116.43 m travelled per game, while the cat maintained higher normalized pressure (1.00) despite lower efficiency (0.53). Qualitative analysis validated smooth state transitions, reliable collision avoidance, and consistent power mode activation, meeting all project objectives for modularity, interpretability, and extensibility.

Future work could include learning-based pursuit strategies, procedural maze generation, and transfer to physical robotic platforms.

References

- [1] “Pac-man.” Wikipedia, The Free Encyclopedia, 1980. Accessed: 2026-01-24.
- [2] Open Robotics, “Gazebo Sim — Get Started Documentation.” <https://gazebo.org/docs/latest/getstarted/>, 2026. Accessed: 2026-02-03.
- [3] Open Robotics, “ROS 2 Jazzy Jalisco Documentation.” <https://docs.ros.org/en/jazzy/index.html>, 2024. Accessed: 2026-02-03.
- [4] S. Macenski, “slam_toolbox: Slam toolbox for lifelong mapping and localization.” https://github.com/SteveMacenski/slam_toolbox, 2026. GitHub repository, LGPL-2.1 licensed software for ROS SLAM.
- [5] ROS Navigation Community, “Navigation2 Documentation.” <https://docs.nav2.org/>, 2026. Documentation for the ROS2 Navigation2 autonomous navigation framework; Accessed: 2026-02-03.
- [6] T. Moore, “robot_localization API Documentation (ROS Melodic).” https://docs.ros.org/en/melodic/api/robot_localization/html/index.html, 2022. Autogenerated API docs; Accessed: 2026-02-03.
- [7] A. Topiwala, P. Inani, and A. Kathpal, “Frontier based exploration for autonomous robot,” 2018.
- [8] Brandonio-c, “Frontier-exploration.” <https://github.com/Brandonio-c/Frontier-Exploration>, 2024. GitHub repository.